

Вопрос 1

1) Логически результат запросов одинаков - в обоих случаях они возвращают одно и то же множество уникальных значений (включая ровно одну строку NULL, если такие есть - и DISTINCT и GROUP BY считают все NULL равными)

Физический план (EXPLAIN) может различаться, это разные операторы и планировщик использует для них разные узлы:

- DISTINCT часто выполняется через узел Unique (обычно в паре с Sort -> «Sort + Unique»), либо, при достаточном work_mem, через HashAggregate.
- GROUP BY без агрегатных функций выполняется через GroupAggregate (требуется отсортированный вход, значит будет Sort) или тоже через HashAggregate.

То есть итоговый набор данных одинаков, разница в производительности.

2) Потому что ASC - это направление сортировки по умолчанию. ORDER BY flight_no и ORDER BY flight_no ASC полностью эквивалентны; ASC писать не обязательно. А строки с разным регистром DISTINCT и GROUP BY считают разными поэтому 'PG0001' и 'pg0001' обе попадут под результат.

Вопрос 2

1) Выручка SUM(amount) - считается по билетам/местам, а не по пассажирам. Два билета = два amount, это корректно: выручка отражает каждую оплаченную перевозку.

Количество пассажиров - если считать COUNT(*), то такой пассажир будет посчитан дважды, потому что в соединении flights -> ticket_flights -> tickets для него образуются две строки. Это и есть «двойной учёт».

2) При INNER JOIN такой рейс выпадет из результата полностью (нет строки в правой таблице - нет строки в результате). Чтобы рейс попал с нулевой выручкой, нужен LEFT JOIN.

Вопрос 3

1) Если город берётся из airports, а аэропорт ни разу не встречается в flights, значит из него не было ни одного рейса - тем более ни одного отменённого. Такой город должен попасть в результат (он удовлетворяет условию «нет ни одного рейса со статусом Cancelled»). И NOT IN, и NOT EXISTS это сделают корректно, при условии, что подзапрос не возвращает NULL.

2) Если подзапрос вернёт хотя бы один NULL, весь NOT IN вернёт пустой результат для всех строк. Причина - семантика NOT IN раскрывается в цепочку <>. В NOT EXISTS нет сравнения со списком значений. Для каждой строки проверяется «существует ли хоть одна строка, где и аэропорт совпал, и статус = Cancelled». NULL-значения status просто не проходят условие status = 'Cancelled' (даёт NULL - строка не подходит в EXISTS), и не «ломают» весь запрос. Поэтому NOT EXISTS предпочтительнее.

Вопрос 4

1) DISTINCT убирает дубли по всем выводимым столбцам.

GROUP BY убирает дубли по указанным столбцам и дополнительно позволяет считать агрегаты.

Планировщик может выполнить оба через Sort + Unique либо через HashAggregate. На практике для одного столбца планы почти не различаются; GROUP BY выигрывает, когда нужно одновременно посчитать что-то (COUNT(*), COUNT(DISTINCT ...)), потому что это делается за один проход. Семантического преимущества «DISTINCT быстрее GROUP BY» в PostgreSQL нет - выбор за тем, нужны ли агрегаты.

2) Если запрос строится как tickets -> ticket_flights -> boardings через INNER JOIN, то пассажир не попадёт в результат: строка в ticket_flights отсутствует, и цепочка INNER JOIN оборвётся.

Если запрос строится от boardings (например, boardings -> tickets), то пассажир попадёт, так как посадочный талон есть.

Вопрос 5

1) SELECT COUNT(*) берёт блокировку уровня ACCESS SHARE и держит её до конца транзакции (COMMIT/ROLLBACK).

ALTER TABLE ADD COLUMN требует блокировку ACCESS EXCLUSIVE - самую сильную, она конфликтует со всеми режимами, включая ACCESS SHARE.

Поэтому команда DDL встанет в очередь и будет ждать, пока завершится транзакция с SELECT.

2) Начиная с PostgreSQL 11, ADD COLUMN с неизменяемым (не-volatile) значением по умолчанию не переписывает таблицу. Значение false - константа, поэтому операция выполняется как изменение только системного каталога (O(1)), без физической записи в каждую строку:

Значение по умолчанию сохраняется в pg_attrdef.

А для существующих строк PostgreSQL хранит так называемое attmissingval в pg_attribute - «виртуальное» значение, которое подставляется при чтении строк, у которых физически нет этого столбца.

3) Обычный CREATE INDEX накладывает блокировку уровня SHARE. Она конфликтует с ROW EXCLUSIVE, которую берут INSERT/UPDATE/DELETE.

Поэтому на время построения индекса запись в таблицу блокируется (чтение/SELECT продолжает работать, так как SHARE не конфликтует с ACCESS SHARE). Синтаксис который стоит использовать:

```
CREATE INDEX CONCURRENTLY idx_flights_status ON flights(status);
```

Компромиссы:

- построение дольше и требует больше ресурсов (два прохода по таблице);
- нельзя выполнять внутри блока транзакции (BEGIN; ... COMMIT;);
- при сбое может остаться невалидный (invalid) индекс, который нужно удалить и пересоздать.

Поэтому CONCURRENTLY применяют не всегда, а только на нагруженных боевых таблицах.

Вопрос 6

1) AVG(actual_departure - scheduled_departure) при actual_departure = NULL.

Если `actual_departure` равен `NULL`, то выражение `actual_departure - scheduled_departure` тоже `NULL`. `AVG` игнорирует `NULL`-значения - такие строки полностью исключаются из расчёта (не считаются ни в числителе суммы, ни в знаменателе количества).

Следствие: среднее вычисляется только по рейсам, которые реально вылетели, а отменённые (без фактического вылета) не учитываются. Ошибки не будет, но итог может быть смещён - решение (исключать отменённые или считать им задержку = 0 / особую метку) должно быть бизнес-правилом.

2) `COUNT(*)` считает все строки.

`COUNT(column)` считает только не-`NULL` значения столбца.

Если на `aircraft_code` стоит ограничение `NOT NULL` (в демо-схеме это внешний ключ на `aircrafts`, поэтому колонка `NOT NULL`), то результаты совпадают.

Различие возникает, когда колонка может содержать `NULL`:

- при обычной таблице, где `aircraft_code` допускает `NULL`;
- либо при `LEFT JOIN`, где для «пустых» строк все колонки правой таблицы становятся `NULL`.

Тогда `COUNT(aircraft_code) < COUNT(*)` - первый пропустит строки с `NULL`, второй посчитает все. Именно так связь «`COUNT(column) <-> ограничение NOT NULL`» и проявляется - наличие `NOT NULL` гарантирует равенство результатов.